Styling chunks

Calepin wraps executed code and its output in a small amount of visual chrome: a tinted box around echoed source, a bordered box around output. Every piece of that chrome is applied by a Typst show rule targeting a labeled element, so you can restyle or remove any of it with plain Typst and no Calepin-specific API.

```
#show <calepin-input>: it => it.body   // strip Calepin's box entirely
```

The labels

Each label marks a `block` whose `body` field holds the bare, unstyled content.

Label	Emitted by	`.body` holds
`<calepin-input>`	Echoed chunk source, and fenced blocks Calepin renders	the syntax-highlighted `raw` element
`<calepin-output>`	Text printed by a chunk (stdout)	the output text
`<calepin-result>`	A value returned by a chunk	the formatted value
`<calepin-warning>`	Warnings raised during execution	the warning text
`<calepin-error>`	Errors raised during execution	the error message

Warnings and errors carry separate labels so a theme can distinguish a chunk that complained from a chunk that failed.

Writing an override

Always reconstruct from `it.body`. Never re-display `it`.

```
#show <calepin-input>: it => it.body                // strip
#show <calepin-input>: it => my-frame(it.body)      // restyle echoed source
#show <calepin-output>: it => box(it.body)          // restyle stdout
#show <calepin-result>: it => box(it.body)          // restyle returned values
#show <calepin-error>: it => text(red, it.body)     // errors, distinct from
warnings
```

Warning

Re-displaying `it` does not replace the default rule. It nests inside it.

This is ordinary Typst show-rule behavior: when a later rule re-emits `it`, the earlier rule fires again on the re-displayed element. So `it => it` changes nothing at all, and `it => my-frame(it)` puts your frame **around** Calepin's box instead of replacing it. Neither produces an error; you simply get the wrong output.

Reconstructing from a field (`it.body`) never re-displays the labeled element, so the default rule has nothing left to fire on and your rule wins.

A show-set rule composes rather than replaces, which is usually what you want:

```
#show raw: set text(size: 9pt)   // resizes the code; Calepin's box stays
```

Removing all chrome at once

Set `theme = "typst"` in your config or front matter. That injects no theme bundle, so nothing installs the default rules and the labeled carriers render bare. Chunks still execute normally, because execution does not depend on the theme.

For theme authors

A theme restyles chunks by writing the same show rules in its `layouts/pdf.typ`. There is nothing to opt into and no Calepin-specific function to call:

```
#show <calepin-input>: it => my-code-frame(it.body)

{{ doc.body }}
```

Because a document body is inlined **after** the theme preamble, a document author's rules are always defined later than a theme's and take precedence, as long as both sides reconstruct from `it.body`.

Stability

The following are stable API and will not change without a deprecation path:

- the five label names,
- the guarantee that each labels a `block` whose `.body` is the bare unstyled content,
- the rule that all default chrome is applied through show rules you can displace.

Everything else about the default appearance (colors, strokes, insets, corner radius, the exact markup `code-block` produces) may change between versions without notice. Style your documents against the labels, not against the defaults.